

Revisão dos Módulos 1 e 2

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Módulo 1 – Fundamentos e Aspectos Organizacionais
 - Conceitos e importância do reuso de software
 - Estado da arte e prática atual do reuso
 - Aspectos gerais, níveis e tipos de reuso
 - Aspectos organizacionais e programas de reuso
 - Institucionalização do reuso e barreiras culturais, técnicas e gerenciais
- Módulo 2 – Engenharia de Domínio e Aplicação
 - Engenharia de domínio: objetivos e etapas
 - Engenharia de aplicação e variabilidades
 - Paradigmas de programação e reutilização
 - Reuso na programação orientada a objetos
 - Conceitos de abstração, herança, composição, acoplamento e coesão
 - Padrões de projeto (GoF) e suas categorias:

Conceitos e Importância do Reuso

- **Definição:** Reuso de software é a utilização de artefatos existentes (código, componentes, requisitos, testes, documentos, etc.) em novos contextos de desenvolvimento.
- **Motivação:** Redução de custos, aumento da produtividade, padronização, confiabilidade e qualidade.
- Relação com Engenharia de Software:
 - Está associado à manutenção, qualidade e produtividade.
 - Alinha-se ao conceito de fábrica de software e à ideia de linhas de produto.

Estado da Arte e da Prática

- Evolução do reuso desde os anos 80 (componentes, frameworks, serviços, microserviços).
- Tendências atuais: ecossistemas de software, repositórios abertos (GitHub, CodeTriage), reuso baseado em APIs, containers e design systems.
- Pergunta para discussão: Como o modelo de ecossistemas de software (ex: Android, React, .NET) potencializa o reuso?

Aspectos Gerais e Níveis de Reuso

- Níveis:
 - Código-fonte (funções, classes, bibliotecas).
 - Componentes e frameworks (camadas reutilizáveis).
 - Arquiteturas e padrões de projeto.
 - Domínios e linhas de produto.
- Tipos:
 - reuso interno (intraempresa)
 - externo (comunidades, open source).

Aspectos Organizacionais

- **Programas de reuso:** políticas internas, catálogo de artefatos, métricas e certificações.
- **Serviços de suporte:** repositórios, ferramentas CASE, frameworks e bibliotecas corporativas.
- **Barreiras:** culturais (resistência), técnicas (integração), gerenciais (falta de apoio).
- Qual barreira ao reuso é mais difícil de superar na prática?

Institucionalização do Reuso

- Criação de centros de reuso e repositórios corporativos.
- Estratégias de incentivo e métricas (nível de maturidade de reuso).
- Importância de documentação, padronização e treinamento.

Engenharia de Domínio

- **Paradigma OO favorece o reuso via:**
 - Abstração → encapsula detalhes.
 - Herança e polimorfismo → permitem variação controlada.
 - Composição → reutilização via agregação em vez de herança direta.

- **Acoplamento e Coesão:**
 - Alta coesão → classe com responsabilidade única.
 - Baixo acoplamento → poucas dependências entre classes.

- **Exemplo prático:** uma classe Pedido que delega o cálculo de frete a um ServicoFrete (baixo acoplamento).

Paradigmas de Programação e Reuso

- **Objetivo:** identificar, modelar e construir artefatos reutilizáveis (requisitos, arquitetura, componentes).
- **Etapas:**
 - Análise de domínio (identificar variabilidades e comunalidades).
 - Projeto de domínio (modelar soluções genéricas).
 - Implementação de domínio (produzir artefatos base).
 - Engenharia de Aplicação: usa os artefatos do domínio para criar sistemas concretos.
- **Exemplo:** família de sistemas de transporte — diferentes apps (ônibus, metrô, táxi) derivados do mesmo núcleo.

Padrões de Projeto (GoF)

- **Propósito:** soluções reutilizáveis para problemas recorrentes.
- **Categorias:**
 - Comportamentais: Strategy, Observer, State, Command, Template Method.
 - Estruturais: Adapter, Decorator, Facade, Composite, Proxy, Builder.
 - Criacionais: Factory Method, Singleton, Builder.

Padrões de Projeto (GoF)

Padrão	Problema Resolvido	Exemplo
Strategy	Várias formas de executar um comportamento	Algoritmos de pagamento diferentes
Factory	Criação de objetos desacoplada	Criar conexões de banco de dados
Adapter	Interfaces incompatíveis	Adaptar API antiga para novo formato
Observer	Notificação entre objetos	Sistema de eventos e listeners
Facade	Simplificar subsistemas complexos	Classe única para acessar múltiplos serviços
Composite	Hierarquia de objetos parte-todo	Menu com submenus
State	Comportamento muda conforme estado	Máquina de vendas
Command	Encapsular ações como objetos	Histórico de comandos (Undo/Redo)

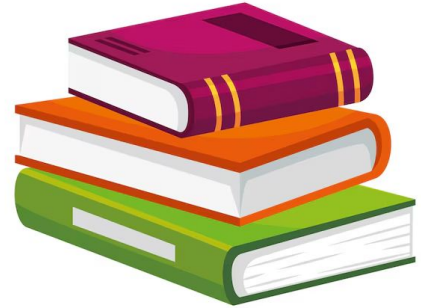
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

